

# NeuroRDViz Modernization — Setup Report

## 1. Repository Cloned and Updated

Cloned the existing repo from <https://github.com/neurord/NeuroRDViz.git> into `/Users/lg002/NeuroRDViz`.

Replaced the old PyQt4/Python 2 `NeuroRDViz.py` with the modernized PySide6 version (from `NeuroRDViz_Working_Morphology.txt`).

Copied the test data file `Model_mglur_spine-CaN.h5` into the project directory.

## 2. Key Changes in the Modernized Code

- **Qt framework:** PyQt4 → PySide6 (Qt6). Compatibility aliases map old `QtGui.*` widget references to their new `QtWidgets.*` locations.
- **HDF5 bytes handling:** All molecule name lookups normalize bytes→str consistently, since h5py returns bytes for string datasets. The old code compared raw bytes directly, which breaks under Python 3.
- **Output set iteration:** `get_mol_info()` now iterates all output sets (not just `outputsets[1:]` and hardcoded keys), handles varying time-step counts across sets, and picks the maximum sample count for molecules that appear in multiple output sets.
- **Voxel volume extraction:** `get_voxel_volumes()` robustly handles the HDF5 grid — tries structured array field 'volume' first, then falls back to a separate dataset, then to uniform 1.0.
- **Time-step mismatch handling:** `get_voxel_molecule_conc()` defensively handles cases where output sets have different numbers of time points (truncates or pads).
- **Animation loop:** The `anim()` generator now updates via `surf.mlab_source.set(scalars=...)` (preferred Mayavi update path) with a fallback to direct unstructured grid modification. Slider updates block signals to prevent feedback loops.
- **Signal connections:** `comboBox.activated[str]` replaced with `comboBox.textActivated` (PySide6-native signal).
- **Dialog API updates:** `QMessageBox.question()` uses `StandardButton` enum flags instead of positional string arguments.

## 3. Environment Setup

Creating a working Python environment on macOS ARM64 (Apple Silicon) required navigating several dependency conflicts:

| Attempt | Approach                                           | Failure                                                                                                                |
|---------|----------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| 1       | pip install everything                             | Mayavi wheel build crashes — pip's VTK wheels trigger an IOKit <code>ENABLE_FIELD_RECOGNITION</code> error             |
| 2       | Conda-forge for everything at once                 | Solver took 10+ minutes and hung                                                                                       |
| 3       | Conda-forge VTK + pip Mayavi + pip PySide6         | Qt library conflict — conda VTK ships its own <code>libQt6Core.dylib</code> , clashing with pip PySide6's bundled core |
| 4       | Conda-forge VTK + conda-forge PySide6 + pip Mayavi | TVTK/VTK version mismatch — Mayavi 4.8.3's generated tvtk wrapper classes don't handle VTK 9.5.2                       |
| 5 ✓     | All from conda-forge: mayavi, h5py, pyside6        | Worked, but required pinning <code>numpy&lt;2</code> (see below)                                                       |

**The `numpy<2` pin:** VTK 9.4.2's `numpy_interface/algorithms.py` references `numpy.in1d`, which was removed in numpy 2.0. Pinning to `numpy 1.26.4` resolves this. This is a known upstream issue; future VTK

releases will fix it.

#### Final working environment:

```
conda create -n neurordviz --strict-channel-priority \
  -c conda-forge python=3.11 mayavi h5py pyside6
conda install -n neurordviz -c conda-forge "numpy<2"
```

| Package  | Version |
|----------|---------|
| Python   | 3.11    |
| VTK      | 9.4.2   |
| Mayavi   | 4.8.3   |
| PySide6  | 6.9.2   |
| numpy    | 1.26.4  |
| h5py     | 3.15.1  |
| traits   | 7.1.0   |
| traitsui | 8.0.0   |

## 4. Verified Working

```
conda activate neurordviz
cd /Users/lg002/NeuroRDViz
python NeuroRDViz.py Model_mglur_spine-CaN.h5
```

The application launches, renders the 36-voxel spine morphology as hexahedral mesh elements (432 edges), populates the molecule dropdown with all 20 species, and is ready for concentration animation on molecule selection.

## 5. New UI Features

### Global Colorbar Range

The concentration colorbar now shows the global min/max across the *entire* animation, not just the current frame. This is locked before the first frame renders and re-locked after every frame update so the color scale remains stable throughout playback.

### Multi-Viewer Architecture

Multiple viewers can run independent animations simultaneously. Each viewer has its own Mayavi scene, molecule dropdown, start/stop buttons, progress bar, and scrub slider.

Getting this working required solving a deep Mayavi scoping bug. Both the global `mlab` module and the scene-specific `viewer.visualization.scene.mlab` route pipeline operations through Mayavi's global engine, which adds new objects to whichever scene was last activated — always the newest viewer. This caused all animation surfaces to render in the most recently created viewer regardless of which viewer's controls were used.

The fix: **never create new pipeline objects during animation**. `Visualization.update_plot()` (fired on `scene.activated`, when the scene IS the active one) now stores `self.ug` and `self.surf`. Animation setup configures the *existing* surface's LUT (colormap, range, scalar bar) and sets concentration scalars directly on the existing unstructured grid. Each frame update modifies `ug.point_data.scalars` in-place and calls `scene.render()` on the correct viewer's scene. A per-viewer `QTimer` drives each animation independently, replacing the old `@mlab.animate` decorator (which also used global figure routing).

## Animation Controls

- **Per-viewer Start/Stop buttons:** Start or stop each viewer's animation independently. Starting viewers at different times produces desynchronized animations.
- **Start All / Stop All:** Global buttons (also in File menu and toolbar) that launch or halt all viewer animations simultaneously. 'Start All' resets all frames to 0 first, ensuring perfect sync.
- **Per-viewer progress bar + scrub slider:** Each viewer shows its own progress bar, time label, and horizontal slider beneath the 3D view. Dragging the slider seeks that viewer independently.
- **Global scrub slider:** A slider at the bottom of the window that, when dragged, moves *all* viewers to the same percentage point — useful for comparing molecules at the same time offset.
- **+ Add Viewer button:** Adds a new viewer from the bottom of the window without using the menu.